

PYTHAI.bankon.oracle — Production Specification

Apache 2.0 — (c) 2026 BANKON — all rights reserved.

Executive Summary

The `PYTHAI.bankon.oracle` is the BANKON-native oracle aggregation and synthetic-feed engine that sits one layer above `PYTHAI.algo.pyth.oracle`. Where the lower contract is a faithful Algorand client of the upstream Pyth Network pull oracle, this upper contract is the policy and aggregation layer that BANKON's own products consume: SATPAY (the HTLC Bitcoin ↔ BKS bridge), the DELTAVERSE Debt Inheritance Protocol on Algorand, the PWAFF wrapped asset factory, the CONCLAVE Treasurer and Risk Counsellors, and any DAIO economic policy that needs a single, deterministic, governance-bound price truth.

It performs four functions that the raw Pyth client cannot. First, it derives the BANKON-native synthetic feeds BKS/USD and BKP/USD that Pyth does not list. Second, it maintains 1-minute, 5-minute, 15-minute, 1-hour and 24-hour time-weighted average prices in ring buffers, so consumers can choose between spot and TWAP. Third, it computes a cross-source median between Pyth, Folks Finance oracle, and BANKON DEX TWAP for the major reference assets, so a single corrupted source cannot mint counterfeit value. Fourth, it enforces a Tessera-tier subscription model with x402-avm metering so that downstream usage funds the AERARIUM treasury and the keeper economy.

The contract is named `bankon.oracle` under the `PYTHAI` NFD namespace on Algorand. The mainnet application identifier is published in `agenticplace.pythai.net/allchain.html` under CAIP-2 `algorand:wGHE2Pwvdvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=`.

Naming and Identity

The NFD record is `bankon.oracle` registered as a subname of the `PYTHAI` parent NFD at `app.nf.domains`. The verified fields on the NFD are `app_id` (the deployed application identifier), `caip2` (`algorand:wGHE2Pwvdvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=`), `upstream_oracle` (the application identifier of `PYTHAI.algo.pyth.oracle`), `aerarium_addr` (the BANKON treasury), `senatus_addr` (the BONAFIDE Senatus 5-of-7 multisig), and `audit_report` (an IPFS CID of the most recent third-party audit). NFD verification ties the human-readable name to the deployed contract and survives any future migration because consumers resolve the name at call time rather than hard-coding the application identifier.

Architectural Overview

The data path begins at the upstream `PYTHAI.algo.pyth.oracle`. Keepers submit Pyth Hermes PNAU updates to the upstream oracle on a schedule driven by Pyth Entropy-randomized

rotation, then call `BankonOracle.ingest()` which reads the freshly-updated raw prices from upstream box storage, appends each one to the relevant TWAP ring buffer, derives the synthetic BKS/USD and BKP/USD prices from the appropriate upstream feeds and the BANKON DEX TWAP, computes the cross-source median where multiple sources exist, evaluates the SATPAY reserve-versus-supply invariant, and writes the resulting `BankonPriceStruct` into the per-feed read box. Consumers then call `get_price_no_older_than`, `get_twap`, or `get_satpay_health` to read attested values. Tessera-tier consumers get sub-second freshness; non-Tessera consumers get a 60-second cache. Every read by a non-Tessera consumer is x402-metered at 0.01 USDC; every write by a keeper is rewarded with a small Algos bounty from the AERARIUM keeper fund proportional to the number of feeds updated.

The trust path is deliberately layered. The upstream `PYTHAI.algo.pyth.oracle` already enforces Wormhole 13-of-19 Guardian verification, Pythnet emitter authentication, Keccak-160 Merkle inclusion against the Wormhole-attested root, per-VAA replay protection, and a configurable deviation envelope. The `BankonOracle` adds median-of-three aggregation for major feeds, TWAP smoothing that defeats single-block manipulation, deviation envelopes against the contract's own TWAP, SATPAY reserve attestation, and BONAFIDE-Senatus 5-of-7 governance over every policy parameter. The cumulative attacker work factor to mint counterfeit BANKON value is the product of compromising Wormhole Guardians, defeating Pyth aggregation, manipulating two independent secondary sources, evading TWAP smoothing, exceeding the deviation envelope, and bypassing Senatus veto — all within the same epoch.

Repository Layout

The repository follows the flat snake_case cypherpunk2048 convention.

```

pythai_bankon_oracle/
├── LICENSE                                     # Apache 2.0
├── README.md
├── pyproject.toml                             # python ^3.12, algorand-python ^3.5
├── contracts/
│   ├── __init__.py
│   ├── bankon_oracle.py                     # main ARC4Contract
│   ├── synthetic_feed_engine.py             # BKS/USD, BKPY/USD derivation
│   ├── twap_buffer.py                       # ring-buffer TWAP for 5 windows
│   ├── median_aggregator.py                 # 3-source median
│   ├── satpay_health.py                     # reserve / supply attestation
│   ├── tessera_subscription.py              # BONAFAIDE Tessera tier gate
│   ├── entropy_keeper.py                    # Pyth Entropy keeper rotation
│   ├── push_dispatcher.py                   # downstream consumer notifications
│   ├── bankon_governance.py                 # Senatus 5-of-7 knobs
│   └── bankon_price_struct.py               # ARC-4 PriceStruct + auxiliary
├── sdk/
│   └── pythai_bankon_oracle_sdk/
│       ├── __init__.py
│       ├── reader.py                        # spot, TWAP, median, health
│       ├── keeper.py                        # automated ingestion loop
│       ├── feeds.py                         # symbol registry incl. synthetic
│       └── subscription.py                  # Tessera tier helpers
├── api/
│   └── mindx_bankon_api/
│       ├── app.py                           # FastAPI + x402-avm middleware
│       ├── routes_read.py
│       ├── routes_subscribe.py
│       ├── routes_governance.py
│       └── mcp_server.py
├── tests/
│   ├── test_synthetic_feeds.py
│   ├── test_twap.py
│   ├── test_median.py
│   ├── test_satpay_health.py
│   ├── test_governance.py
│   └── test_end_to_end.py
└── deploy/
    ├── deploy_testnet.py
    └── deploy_mainnet.py

```

Every file carries the standard header.

```
python
```

```
# Copyright (c) 2026 BANKON – all rights reserved.  
# Licensed under the Apache License, Version 2.0.  
# SPDX-License-Identifier: Apache-2.0
```

Core Data Structures

The `BankonPriceStruct` extends the upstream Pyth `PriceStruct` with median and TWAP fields and a source-attribution bitmap.

```
python
```

```

# contracts/bankon_price_struct.py
# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""ARC-4 BankonPriceStruct and supporting types."""
from algopy import Bytes, UInt64, arc4

# 32-byte BANKON feed identifier (BTC/USD, BKS/USD, BKP/USD, ...).
FeedId = Bytes

class BankonPriceStruct(arc4.Struct, frozen=True):
    """Aggregated, TWAP-smoothed, median-aggregated BANKON price."""

    spot_price: arc4.UInt64
    spot_negative: arc4.Bool
    median_price: arc4.UInt64
    median_negative: arc4.Bool
    twap_1m: arc4.UInt64
    twap_5m: arc4.UInt64
    twap_15m: arc4.UInt64
    twap_1h: arc4.UInt64
    twap_24h: arc4.UInt64
    expo: arc4.UInt64
    expo_negative: arc4.Bool
    publish_time: arc4.UInt64
    conf: arc4.UInt64
    # Bit 0: Pyth, Bit 1: Folks Oracle, Bit 2: BANKON DEX TWAP,
    # Bit 3: synthetic derivation, Bit 4: SATPAY reserve attestation.
    source_bitmap: arc4.UInt64
    num_sources: arc4.UInt64

class TwapSlot(arc4.Struct, frozen=True):
    """Single ring-buffer slot."""
    timestamp: arc4.UInt64
    price: arc4.UInt64
    negative: arc4.Bool

class SatpayHealth(arc4.Struct, frozen=True):
    reserve_satoshis: arc4.UInt64
    circulating_bks: arc4.UInt64
    last_attestation: arc4.UInt64
    healthy: arc4.Bool

```

```
coverage_bps: arc4.UInt64 # 10_000 = 100%
```

```
class TesseraSubscription(arc4.Struct, frozen=True):  
    tessera_id: arc4.UInt64  
    tier: arc4.UInt64 # 0 = none, 1 = standard, 2 = premium, 3 = institutional  
    expires_at: arc4.UInt64  
    paid_usdc: arc4.UInt64
```

TWAP Buffer

The TWAP buffer is a ring of fixed size per window. Five windows are maintained per feed: 1m (12 slots at 5-second resolution), 5m (10 slots at 30-second resolution), 15m (15 slots at 60-second resolution), 1h (12 slots at 5-minute resolution), and 24h (24 slots at 1-hour resolution). The buffer is keyed by `feed_id || window_id` and stored as a `BoxMap` of fixed-length byte arrays so that a write replaces exactly one slot rather than rewriting the entire buffer.

```
python
```

```

# contracts/twap_buffer.py
# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""Ring-buffer TWAP storage for five windows per feed."""
from algopy import Bytes, Global, UInt64, op, subroutine

# (window_id, slot_count, slot_seconds)
WINDOW_1M = (UInt64(1), UInt64(12), UInt64(5))
WINDOW_5M = (UInt64(2), UInt64(10), UInt64(30))
WINDOW_15M = (UInt64(3), UInt64(15), UInt64(60))
WINDOW_1H = (UInt64(4), UInt64(12), UInt64(300))
WINDOW_24H = (UInt64(5), UInt64(24), UInt64(3600))

# Each slot: 8 bytes timestamp | 8 bytes price = 16 bytes.
SLOT_SIZE = UInt64(16)

@subroutine
def buffer_key(feed_id: Bytes, window_id: UInt64) -> Bytes:
    return Bytes(b"tw:") + feed_id + op.itob(window_id)

@subroutine
def append_slot(
    feed_id: Bytes,
    window_id: UInt64,
    slot_count: UInt64,
    price: UInt64,
) -> None:
    """Append a (timestamp, price) slot to the ring, overwriting the oldest."""
    key = buffer_key(feed_id, window_id)
    if not op.Box.length(key)[1]:
        op.Box.create(key, slot_count * SLOT_SIZE)
    # Find the oldest slot (smallest timestamp).
    oldest_idx = UInt64(0)
    oldest_ts = op.btoi(op.Box.extract(key, 0, 8))
    i = UInt64(1)
    while i < slot_count:
        ts = op.btoi(op.Box.extract(key, i * SLOT_SIZE, 8))
        if ts < oldest_ts:
            oldest_ts = ts
            oldest_idx = i
        i += 1
    new_slot = op.itob(Global.latest_timestamp) + op.itob(price)

```

```
op.Box.replace(key, oldest_idx * SLOT_SIZE, new_slot)
```

```
@subroutine
```

```
def compute_twap(
```

```
    feed_id: Bytes,
```

```
    window_id: UInt64,
```

```
    slot_count: UInt64,
```

```
    window_seconds: UInt64,
```

```
) -> UInt64:
```

```
    """Time-weighted mean over the most recent window_seconds, samples-only."""
```

```
    key = buffer_key(feed_id, window_id)
```

```
    if not op.Box.length(key)[1]:
```

```
        return UInt64(0)
```

```
    cutoff = Global.latest_timestamp - window_seconds
```

```
    sum_price = UInt64(0)
```

```
    count = UInt64(0)
```

```
    i = UInt64(0)
```

```
    while i < slot_count:
```

```
        ts = op.btoi(op.Box.extract(key, i * SLOT_SIZE, 8))
```

```
        if ts >= cutoff and ts != 0:
```

```
            sum_price += op.btoi(op.Box.extract(key, i * SLOT_SIZE + 8, 8))
```

```
            count += 1
```

```
        i += 1
```

```
    if count == 0:
```

```
        return UInt64(0)
```

```
    return sum_price // count
```

This is a sample-weighted mean rather than a true time-weighted mean to keep the AVM opcode budget bounded. For BANKON's purposes the difference is negligible because the keeper-driven sampling interval is nearly uniform.

Median Aggregator

The median aggregator accepts three candidate prices and returns the middle value. For two-source feeds it returns the mean; for one-source feeds it returns that source verbatim.

```
python
```

```

# contracts/median_aggregator.py
# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""Three-source median with two- and one-source fallbacks."""
from algopy import UInt64, subroutine

@subroutine
def median_of_three(a: UInt64, b: UInt64, c: UInt64) -> UInt64:
    # The middle of three values: (a+b+c) - max - min.
    mn = a
    mx = a
    if b < mn:
        mn = b
    if b > mx:
        mx = b
    if c < mn:
        mn = c
    if c > mx:
        mx = c
    return (a + b + c) - mn - mx

@subroutine
def aggregate(
    a: UInt64, a_present: bool,
    b: UInt64, b_present: bool,
    c: UInt64, c_present: bool,
) -> UInt64:
    if a_present and b_present and c_present:
        return median_of_three(a, b, c)
    if a_present and b_present:
        return (a + b) // 2
    if a_present and c_present:
        return (a + c) // 2
    if b_present and c_present:
        return (b + c) // 2
    if a_present:
        return a
    if b_present:
        return b
    return c

```

Synthetic Feed Engine

The synthetic feed engine derives BKS/USD and BKPY/USD from upstream sources.

BKS (BANKON SATOSHI) has a hard-coded supply of 2,100,000,000,000,000 units, one per satoshi. By definition 1 BKS represents 1 Bitcoin satoshi, so BKS/USD is mechanically $\frac{\text{BTC/USD}}{10^8}$. There is no market price discovery for BKS because the SATPAY bridge enforces a 1:1 reserve invariant — circulating BKS supply cannot exceed reserved satsoshis. The oracle therefore derives BKS/USD deterministically from the Pyth BTC/USD feed and the exponent shift.

BKPY (BANKON PYTHAI) has a supply of 10,000 units and is a genuine market-priced asset. Its USD price is derived from the BANKON DEX TWAP for the BKPY/USDC pair, anchored to dollars via Pyth USDC/USD. The DEX TWAP is read from a BANKON-owned DEX TWAP oracle whose application identifier is recorded in `bankon_governance.bankon_dex_twap_app_id`.

```
python
```

```

# contracts/synthetic_feed_engine.py
# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""Deterministic synthetic feed derivation for BKS/USD and BKPY/USD."""
from algopy import Bytes, UInt64, arc4, itxn, op, subroutine

# Pyth feed identifiers (32 bytes each).
BTC_USD_PYTH = Bytes(
    bytes.fromhex("e62df6c8b4a85fe1a67db44dc12de5db330f7ac66b72dc658afedf0f4a415b43")
)
USDC_USD_PYTH = Bytes(
    bytes.fromhex("eaa020c61cc479712813461ce153894a96a6c00b21ed0cfc2798d1f9a9e9c94a")
)

# BANKON-native synthetic identifiers (32 bytes, BANKON-controlled namespace).
BKS_USD_BANKON = Bytes(b"BANKON:BKS/USD\x00".ljust(32, b"\x00"))
BKPY_USD_BANKON = Bytes(b"BANKON:BKPY/USD".ljust(32, b"\x00"))

# Hard-coded BANKON SATOSHI supply:  $2.1 * 10^{15}$ .
BKS_SUPPLY = UInt64(2_100_000_000_000_000)
SATS_PER_BTC = UInt64(100_000_000)

@subroutine
def derive_bks_usd(btc_usd_price: UInt64, btc_usd_expo_abs: UInt64) -> tuple[UInt64,
    """Returns (bks_price_micros, expo_abs).

    BKS/USD = BTC/USD /  $10^8$ . To preserve precision we keep BTC/USD in its
    native Pyth scaling (typically expo = -8) and simply subtract 8 from the
    exponent, i.e. multiply the exponent absolute value by  $10^8$  in denominator.
    Equivalently:  $\text{bks\_price} = \text{btc\_usd\_price}$ ,  $\text{expo} = \text{btc\_usd\_expo} - 8$ .
    """
    return (btc_usd_price, btc_usd_expo_abs + UInt64(8))

@subroutine
def derive_bkpy_usd(
    bkpy_usdc_twap: UInt64,          # BKPY price in USDC, scaled  $10^6$ 
    bkpy_usdc_expo_abs: UInt64,      # typically 6
    usdc_usd_price: UInt64,          # USDC/USD from Pyth, scaled  $10^8$ 
    usdc_usd_expo_abs: UInt64,      # typically 8
) -> tuple[UInt64, UInt64]:
    """BKPY/USD = (BKPY/USDC) * (USDC/USD).

```

```
Combined exponent is the sum of the two component exponents.
```

```
"""
```

```
combined = bkpy_usdc_twap * usdc_usd_price
```

```
combined_expo = bkpy_usdc_expo_abs + usdc_usd_expo_abs
```

```
return (combined, combined_expo)
```

```
@subroutine
```

```
def read_bkpy_usdc_twap(dex_twap_app_id: UInt64) -> UInt64:
```

```
    """Read the BKPY/USDC TWAP from the BANKON DEX TWAP application."""
```

```
    result = itxn.ApplicationCall(
```

```
        app_id=dex_twap_app_id,
```

```
        app_args=(Bytes(b"get_twap_15m"), Bytes(b"BKPY/USDC")),
```

```
        fee=0,
```

```
    ).submit()
```

```
    return op.btoi(result.last_log)
```

SATPAY Health

SATPAY maintains a strict reserve-only invariant: circulating BKS supply must never exceed reserved Bitcoin satoshis held in the SATPAY HTLC reserve. The `satpay_health` module attests this continuously by reading the reserve-attestation root (updated by the off-chain reserve attester with a Senatus-rooted signature) and the circulating supply (from the BANKON SATOSHI ASA application).

```
python
```

```

# contracts/satpay_health.py
# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""SATPAY reserve-versus-supply invariant attestation."""
from algopy import Bytes, Global, UInt64, arc4, itxn, op, subroutine

from .bankon_price_struct import SatpayHealth

@subroutine
def compute_health(
    reserve_satoshis: UInt64,
    circulating_bks: UInt64,
    breach_threshold_bps: UInt64, # e.g. 9_900 = 99% coverage required
) -> SatpayHealth:
    if circulating_bks == 0:
        return SatpayHealth(
            reserve_satoshis=arc4.UInt64(reserve_satoshis),
            circulating_bks=arc4.UInt64(0),
            last_attestation=arc4.UInt64(Global.latest_timestamp),
            healthy=arc4.Bool(True),
            coverage_bps=arc4.UInt64(10_000),
        )
    coverage_bps = (reserve_satoshis * UInt64(10_000)) // circulating_bks
    healthy = coverage_bps >= breach_threshold_bps
    return SatpayHealth(
        reserve_satoshis=arc4.UInt64(reserve_satoshis),
        circulating_bks=arc4.UInt64(circulating_bks),
        last_attestation=arc4.UInt64(Global.latest_timestamp),
        healthy=arc4.Bool(healthy),
        coverage_bps=arc4.UInt64(coverage_bps),
    )

@subroutine
def read_circulating_bks(bks_asa_id: UInt64) -> UInt64:
    """Read circulating supply = total supply - reserve-account holdings."""
    return op.AssetParam.get_total(bks_asa_id)[0]

```

The off-chain attester publishes the reserve root via the same Pyth pull pattern: it forms a structured message containing {reserve_satoshis, btc_block_height, btc_block_hash, attester_signature}, signs with the BONAFIDE Senatus key, and the BANKON oracle accepts

it through a Senatus-only `set_satpay_reserves` method. The signature check is done at the application call boundary by enforcing `Txn.sender == senatus_addr`.

Tessera Subscription

Tessera subscriptions gate sub-second freshness and access to TWAP/median feeds. The contract verifies a Tessera holder by inner-app-calling the BONAFIDE Tessera ASA balance check on the caller's account; a non-zero balance plus a non-expired subscription record grants access.

```
python

# contracts/tessera_subscription.py
# (c) 2026 BANKON — all rights reserved. Apache-2.0.
"""BONAFIDE Tessera subscription tier verification."""
from algopy import Account, Bytes, Global, UInt64, arc4, op, subroutine

from .bankon_price_struct import TesseraSubscription

# Pricing in micro-USDC (10^-6 USD). Quarterly billing.
TIER_STANDARD_QUARTER = UInt64(50_000_000)      # 50 USDC / quarter
TIER_PREMIUM_QUARTER = UInt64(250_000_000)      # 250 USDC / quarter
TIER_INSTITUTIONAL_QUARTER = UInt64(2_500_000_000) # 2,500 USDC / quarter

QUARTER_SECONDS = UInt64(90 * 24 * 3600)

@subroutine
def verify_tier(
    sub: TesseraSubscription,
    required_tier: UInt64,
    tessera_asa_id: UInt64,
    holder: Account,
) -> bool:
    if sub.tier.native < required_tier:
        return False
    if sub.expires_at.native < Global.latest_timestamp:
        return False
    balance = op.AssetHoldingGet.asset_balance(holder, tessera_asa_id)[0]
    return balance > 0
```

A holder activates by sending a USDC payment in an atomic group with

`activate_subscription(tier)`; the oracle records expiry as `now + QUARTER_SECONDS *`

`num_quarters` and routes the payment to AERARIUM. Renewal extends rather than resets.

Pyth Entropy Keeper Rotation

The keeper that submits the next ingestion call is selected pseudo-randomly via Pyth Entropy to prevent any single party from front-running BANKON-oracle-driven liquidations or arbitrage. Each ingestion round draws a fresh entropy reveal; the keeper whose address hash modulo the registered keeper set matches the entropy value is the only one whose call is accepted for that round. All registered keepers receive a small bounty for being eligible; the round winner receives a larger bounty for actually executing.

python

```

# contracts/entropy_keeper.py
# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""Pyth Entropy-driven keeper selection for ingestion rounds."""
from algopy import Account, Bytes, Global, UInt64, arc4, op, subroutine

@subroutine
def is_authorized_keeper(
    caller: Account,
    entropy_value: Bytes,          # 32 bytes from Pyth Entropy
    keeper_set_root: Bytes,        # Merkle root of registered keeper addresses
    proof: Bytes,                  # Merkle inclusion proof for caller
    proof_len: UInt64,
    set_size: UInt64,
) -> bool:
    # 1. Caller must be in the registered keeper Merkle set.
    leaf = op.sha512_256(caller.bytes)
    current = leaf
    i = UInt64(0)
    while i < proof_len:
        sibling = op.extract(proof, i * UInt64(32), UInt64(32))
        if current < sibling:
            current = op.sha512_256(current + sibling)
        else:
            current = op.sha512_256(sibling + current)
        i += 1
    if current != keeper_set_root:
        return False
    # 2. Caller's slot must match the entropy-derived slot for this round.
    selected_slot = op.btoi(op.extract(entropy_value, 0, 8)) % set_size
    caller_slot = op.btoi(op.extract(leaf, 0, 8)) % set_size
    return selected_slot == caller_slot

```

In testnet operation the Pyth Entropy contract has not yet shipped on Algorand; until then the contract falls back to `op.Global.latest_timestamp` modulated by a Senatus-rotated seed. This is documented in the caveats.

Push Dispatcher

Downstream consumers (PWAFF, DELTAVERSE, CONCLAVE) can register for push notifications when a feed's spot price deviates from its 5-minute TWAP by more than a configurable bps threshold. The dispatcher inner-app-calls the registered consumer with the new price and the

deviation magnitude, allowing the consumer to take protective action (pause minting, trigger liquidation, request Senatus veto).

```
python
```

```
# contracts/push_dispatcher.py
# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""Push notification dispatcher for downstream BANKON consumers."""
from algopy import Bytes, UInt64, arc4, itxn, op, subroutine

@subroutine
def dispatch_deviation_alert(
    consumer_app_id: UInt64,
    feed_id: Bytes,
    spot: UInt64,
    twap_5m: UInt64,
    deviation_bps: UInt64,
) -> None:
    itxn.ApplicationCall(
        app_id=consumer_app_id,
        app_args=(
            Bytes(b"on_deviation_alert"),
            feed_id,
            op.itob(spot),
            op.itob(twap_5m),
            op.itob(deviation_bps),
        ),
        fee=0,
    ).submit()

@subroutine
def compute_deviation_bps(spot: UInt64, twap: UInt64) -> UInt64:
    if twap == 0:
        return UInt64(0)
    diff = (spot - twap) if spot > twap else (twap - spot)
    return (diff * UInt64(10_000)) // twap
```

Governance

All policy parameters are gated by the BONAFIDE Senatus 5-of-7 multisig. Governance is intentionally minimal: the contract is designed to be operationally autonomous once configured,

with Senatus acting only on policy drift or emergency.

python

```

# contracts/bankon_governance.py
# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""BONAFIDE Senatus 5-of-7 governance knobs for BankonOracle."""
from algopy import Account, ARC4Contract, GlobalState, Txn, UInt64, arc4
from algopy.arc4 import abimethod

class BankonGovernance(ARC4Contract):
    def __init__(self) -> None:
        # Identity and treasury.
        self.senatus = GlobalState(Account, key="senatus")
        self.aerarium = GlobalState(Account, key="aerarium")
        self.paused = GlobalState(UInt64(0), key="paused")
        # Upstream wiring.
        self.upstream_pyth_app_id = GlobalState(UInt64, key="up_pyth")
        self.folks_oracle_app_id = GlobalState(UInt64, key="folks")
        self.bankon_dex_twap_app_id = GlobalState(UInt64, key="dex_twap")
        self.bks_asa_id = GlobalState(UInt64, key="bks_asa")
        self.tessera_asa_id = GlobalState(UInt64, key="tessera_asa")
        # Policy.
        self.deviation_envelope_bps = GlobalState(UInt64(2_500), key="dev_bps")
        self.satpay_breach_threshold_bps = GlobalState(UInt64(9_900), key="satpay_bp
        self.alert_threshold_bps = GlobalState(UInt64(500), key="alert_bps")
        self.read_fee_microusdc = GlobalState(UInt64(10_000), key="read_fee")
        self.keeper_bounty_micros = GlobalState(UInt64(50_000), key="keeper_bnty")

    def _only_senatus(self) -> None:
        assert Txn.sender == self.senatus.value, "SENATUS_ONLY"

    @abimethod
    def set_pause(self, value: arc4.Bool) -> None:
        self._only_senatus()
        self.paused.value = UInt64(1) if value.native else UInt64(0)

    @abimethod
    def set_upstream_pyth(self, app_id: arc4.UInt64) -> None:
        self._only_senatus()
        self.upstream_pyth_app_id.value = app_id.native

    @abimethod
    def set_folks_oracle(self, app_id: arc4.UInt64) -> None:
        self._only_senatus()
        self.folks_oracle_app_id.value = app_id.native

```

```

@abimethod
def set_dex_twap(self, app_id: arc4.UInt64) -> None:
    self._only_senatus()
    self.bankon_dex_twap_app_id.value = app_id.native

@abimethod
def set_satpay_breach_threshold(self, bps: arc4.UInt64) -> None:
    self._only_senatus()
    assert bps.native <= 10_000, "BPS_CAP"
    self.satpay_breach_threshold_bps.value = bps.native

@abimethod
def set_alert_threshold(self, bps: arc4.UInt64) -> None:
    self._only_senatus()
    self.alert_threshold_bps.value = bps.native

@abimethod
def rotate_senatus(self, new_senatus: arc4.Address) -> None:
    self._only_senatus()
    self.senatus.value = new_senatus.native

```

Main Contract

The `BankonOracle` orchestrates ingestion, aggregation, derivation, dispatch and reads.

python

```

# contracts/bankon_oracle.py
# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""PYTHAI.bankon.oracle – main contract."""
from algopy import (
    Account, BoxMap, Bytes, Global, GlobalState, Txn, UInt64, arc4, gtxn, itxn,
    op, subroutine,
)
from algopy.arc4 import abimethod

from .bankon_governance import BankonGovernance
from .bankon_price_struct import (
    BankonPriceStruct, FeedId, SatpayHealth, TesseraSubscription,
)
from .median_aggregator import aggregate
from .push_dispatcher import compute_deviation_bps, dispatch_deviation_alert
from .satpay_health import compute_health, read_circulating_bks
from .synthetic_feed_engine import (
    BKPY_USD_BANKON, BKS_USD_BANKON, BTC_USD_PYTH, USDC_USD_PYTH,
    derive_bkpy_usd, derive_bks_usd, read_bkpy_usdc_twap,
)
from .tessera_subscription import (
    QUARTER_SECONDS, TIER_INSTITUTIONAL_QUARTER, TIER_PREMIUM_QUARTER,
    TIER_STANDARD_QUARTER, verify_tier,
)
from .twap_buffer import (
    WINDOW_1H, WINDOW_1M, WINDOW_5M, WINDOW_15M, WINDOW_24H,
    append_slot, compute_twap,
)

USDC_ASA_ID = UInt64(31_566_704)

class BankonOracle(BankonGovernance):
    def __init__(self) -> None:
        super().__init__()
        self.prices = BoxMap(FeedId, BankonPriceStruct, key_prefix="px:")
        self.satpay_reserves = GlobalState(UInt64(0), key="satpay_res")
        self.satpay_health = GlobalState(SatpayHealth, key="satpay_h")
        self.subscriptions = BoxMap(Account, TesseraSubscription, key_prefix="sub:")
        self.deviation_alert_consumers = BoxMap(FeedId, UInt64, key_prefix="dac:")

# ----- ingestion

```

```

@abimethod
def ingest(self, pyth_feed_ids: arc4.DynamicArray[arc4.StaticArray[arc4.Byte, 32])
    """Keeper-driven ingestion. Reads upstream Pyth prices, derives synthetics,
    aggregates median, updates TWAP buffers, dispatches alerts, pays bounty."""
    assert self.paused.value == 0, "PAUSED"

    n = pyth_feed_ids.length
    i = UInt64(0)
    while i < n:
        feed_id = pyth_feed_ids[i].bytes
        self._ingest_one(feed_id)
        i += 1

    # Derive synthetic feeds.
    self._ingest_bks()
    self._ingest_bkpy()

    # Refresh SATPAY health.
    self._refresh_satpay_health()

    # Pay keeper bounty.
    itxn.Payment(
        receiver=Txn.sender,
        amount=self.keeper_bounty_micros.value * n,
        fee=0,
    ).submit()

@subroutine
def _ingest_one(self, feed_id: Bytes) -> None:
    # 1. Read upstream Pyth price.
    pyth_price, pyth_expo, pyth_conf, pyth_pubtime = self._read_upstream_pyth(feed_id)

    # 2. Read Folks oracle if configured (optional secondary).
    folks_price, folks_present = self._read_folks(feed_id)

    # 3. Read BANKON DEX TWAP if a DEX pair exists for this feed.
    dex_price, dex_present = self._read_dex_twap(feed_id)

    # 4. Median aggregate.
    median = aggregate(pyth_price, True, folks_price, folks_present, dex_price,

    # 5. TWAP buffer append.
    append_slot(feed_id, WINDOW_1M[0], WINDOW_1M[1], median)
    append_slot(feed_id, WINDOW_5M[0], WINDOW_5M[1], median)

```

```

append_slot(feed_id, WINDOW_15M[0], WINDOW_15M[1], median)
append_slot(feed_id, WINDOW_1H[0], WINDOW_1H[1], median)
append_slot(feed_id, WINDOW_24H[0], WINDOW_24H[1], median)

# 6. Compute TWAPs.
twap_1m = compute_twap(feed_id, WINDOW_1M[0], WINDOW_1M[1], UInt64(60))
twap_5m = compute_twap(feed_id, WINDOW_5M[0], WINDOW_5M[1], UInt64(300))
twap_15m = compute_twap(feed_id, WINDOW_15M[0], WINDOW_15M[1], UInt64(900))
twap_1h = compute_twap(feed_id, WINDOW_1H[0], WINDOW_1H[1], UInt64(3600))
twap_24h = compute_twap(feed_id, WINDOW_24H[0], WINDOW_24H[1], UInt64(86400))

# 7. Deviation envelope check vs 5m TWAP.
if twap_5m > 0:
    dev_bps = compute_deviation_bps(median, twap_5m)
    assert dev_bps <= self.deviation_envelope_bps.value, "DEVIATION_ENVELOPE"

# 8. Dispatch alert if above alert threshold.
if dev_bps >= self.alert_threshold_bps.value:
    self._dispatch_alert(feed_id, median, twap_5m, dev_bps)

# 9. Source bitmap.
bitmap = UInt64(1) # Pyth always present
n_sources = UInt64(1)
if folks_present:
    bitmap = bitmap | UInt64(2)
    n_sources += 1
if dex_present:
    bitmap = bitmap | UInt64(4)
    n_sources += 1

# 10. Write composite price struct.
self.prices[feed_id] = BankonPriceStruct(
    spot_price=arc4.UInt64(pyth_price),
    spot_negative=arc4.Bool(False),
    median_price=arc4.UInt64(median),
    median_negative=arc4.Bool(False),
    twap_1m=arc4.UInt64(twap_1m),
    twap_5m=arc4.UInt64(twap_5m),
    twap_15m=arc4.UInt64(twap_15m),
    twap_1h=arc4.UInt64(twap_1h),
    twap_24h=arc4.UInt64(twap_24h),
    expo=arc4.UInt64(pyth_expo),
    expo_negative=arc4.Bool(True),
    publish_time=arc4.UInt64(pyth_pubtime),
    conf=arc4.UInt64(pyth_conf),

```

```

        source_bitmap=arc4.UInt64(bitmap),
        num_sources=arc4.UInt64(n_sources),
    )

```

@subroutine

```

def _ingest_bks(self) -> None:
    btc = self.prices[BTC_USD_PYTH]
    bks_price, bks_expo = derive_bks_usd(btc.spot_price.native, btc.expo.native)
    self.prices[BKS_USD_BANKON] = BankonPriceStruct(
        spot_price=arc4.UInt64(bks_price),
        spot_negative=arc4.Bool(False),
        median_price=arc4.UInt64(bks_price),
        median_negative=arc4.Bool(False),
        twap_1m=arc4.UInt64(bks_price),
        twap_5m=arc4.UInt64(bks_price),
        twap_15m=arc4.UInt64(bks_price),
        twap_1h=arc4.UInt64(bks_price),
        twap_24h=arc4.UInt64(bks_price),
        expo=arc4.UInt64(bks_expo),
        expo_negative=arc4.Bool(True),
        publish_time=arc4.UInt64(Global.latest_timestamp),
        conf=arc4.UInt64(btc.conf.native), # inherits BTC confidence
        source_bitmap=arc4.UInt64(8),      # bit 3 = synthetic
        num_sources=arc4.UInt64(1),
    )

```

@subroutine

```

def _ingest_bkpy(self) -> None:
    usdc = self.prices[USDC_USD_PYTH]
    bkpy_usdc = read_bkpy_usdc_twap(self.bankon_dex_twap_app_id.value)
    bkpy_price, bkpy_expo = derive_bkpy_usd(
        bkpy_usdc, UInt64(6), usdc.spot_price.native, usdc.expo.native,
    )
    # Append into TWAP buffers.
    append_slot(BKPY_USD_BANKON, WINDOW_5M[0], WINDOW_5M[1], bkpy_price)
    append_slot(BKPY_USD_BANKON, WINDOW_15M[0], WINDOW_15M[1], bkpy_price)
    append_slot(BKPY_USD_BANKON, WINDOW_1H[0], WINDOW_1H[1], bkpy_price)
    twap_5m = compute_twap(BKPY_USD_BANKON, WINDOW_5M[0], WINDOW_5M[1], UInt64(3))
    twap_15m = compute_twap(BKPY_USD_BANKON, WINDOW_15M[0], WINDOW_15M[1], UInt64(3))
    twap_1h = compute_twap(BKPY_USD_BANKON, WINDOW_1H[0], WINDOW_1H[1], UInt64(3))
    self.prices[BKPY_USD_BANKON] = BankonPriceStruct(
        spot_price=arc4.UInt64(bkpy_price),
        spot_negative=arc4.Bool(False),
        median_price=arc4.UInt64(bkpy_price),

```

```

        median_negative=arc4.Bool(False),
        twap_1m=arc4.UInt64(bkpy_price),
        twap_5m=arc4.UInt64(twap_5m),
        twap_15m=arc4.UInt64(twap_15m),
        twap_1h=arc4.UInt64(twap_1h),
        twap_24h=arc4.UInt64(twap_1h),
        expo=arc4.UInt64(bkpy_expo),
        expo_negative=arc4.Bool(True),
        publish_time=arc4.UInt64(Global.latest_timestamp),
        conf=arc4.UInt64(0),
        source_bitmap=arc4.UInt64(8 | 4),    # synthetic + DEX
        num_sources=arc4.UInt64(2),
    )

@subroutine
def _refresh_satpay_health(self) -> None:
    circulating = read_circulating_bks(self.bks_asa_id.value)
    h = compute_health(
        self.satpay_reserves.value,
        circulating,
        self.satpay_breach_threshold_bps.value,
    )
    self.satpay_health.value = h

@abimethod
def set_satpay_reserves(self, satoshis: arc4.UInt64) -> None:
    """Senatus-only: posts the latest off-chain attested reserve count."""
    self._only_senatus()
    self.satpay_reserves.value = satoshis.native
    self._refresh_satpay_health()

# ----- reads

@abimethod(readonly=True)
def get_price_no_older_than(
    self,
    feed_id: arc4.StaticArray[arc4.Byte, 32],
    max_age: arc4.UInt64,
) -> BankonPriceStruct:
    p = self.prices[feed_id.bytes]
    assert Global.latest_timestamp - p.publish_time.native <= max_age.native, "S
    return p

@abimethod(readonly=True)

```

```

def get_twap(
    self,
    feed_id: arc4.StaticArray[arc4.Byte, 32],
    window: arc4.UInt64,
) -> arc4.UInt64:
    p = self.prices[feed_id.bytes]
    if window.native == 60:
        return p.twap_1m
    if window.native == 300:
        return p.twap_5m
    if window.native == 900:
        return p.twap_15m
    if window.native == 3600:
        return p.twap_1h
    if window.native == 86400:
        return p.twap_24h
    assert False, "WINDOW_UNKNOWN"

@abimethod(readonly=True)
def get_satpay_health(self) -> SatpayHealth:
    return self.satpay_health.value

# ----- subscriptions

@abimethod
def activate_subscription(
    self,
    usdc_payment: gtxn.AssetTransferTransaction,
    tier: arc4.UInt64,
    quarters: arc4.UInt64,
) -> None:
    assert usdc_payment.xfer_asset.id == USDC_ASA_ID, "FEE_ASSET"
    assert usdc_payment.asset_receiver == self.aerarium.value, "FEE_RECEIVER"
    balance = op.AssetHoldingGet.asset_balance(Txn.sender, self.tessera_asa_id.v)
    assert balance > 0, "TESSERA_REQUIRED"

    if tier.native == 1:
        cost = TIER_STANDARD_QUARTER * quarters.native
    elif tier.native == 2:
        cost = TIER_PREMIUM_QUARTER * quarters.native
    elif tier.native == 3:
        cost = TIER_INSTITUTIONAL_QUARTER * quarters.native
    else:
        assert False, "TIER_INVALID"

```

```

assert usdc_payment.asset_amount >= cost, "INSUFFICIENT_FEE"

existing_expiry = UInt64(0)
if Txn.sender in self.subscriptions:
    existing_expiry = self.subscriptions[Txn.sender].expires_at.native
base = (
    existing_expiry
    if existing_expiry > Global.latest_timestamp
    else Global.latest_timestamp
)
new_expiry = base + QUARTER_SECONDS * quarters.native

self.subscriptions[Txn.sender] = TesseraSubscription(
    tessera_id=arc4.UInt64(self.tessera_asa_id.value),
    tier=tier,
    expires_at=arc4.UInt64(new_expiry),
    paid_usdc=arc4.UInt64(cost),
)

@abimethod(readonly=True)
def get_subscription(self, holder: arc4.Address) -> TesseraSubscription:
    return self.subscriptions[holder.native]

# ----- deviation consumers

@abimethod
def register_deviation_consumer(
    self,
    feed_id: arc4.StaticArray[arc4.Byte, 32],
    consumer_app_id: arc4.UInt64,
) -> None:
    self._only_senatus()
    self.deviation_alert_consumers[feed_id.bytes] = consumer_app_id.native

@subroutine
def _dispatch_alert(
    self, feed_id: Bytes, spot: UInt64, twap_5m: UInt64, dev_bps: UInt64,
) -> None:
    if feed_id in self.deviation_alert_consumers:
        consumer = self.deviation_alert_consumers[feed_id]
        dispatch_deviation_alert(consumer, feed_id, spot, twap_5m, dev_bps)

# ----- upstream reads

```

```
@subroutine
```

```
def _read_upstream_pyth(self, feed_id: Bytes) -> tuple[UInt64, UInt64, UInt64, UInt64]:
    result = itxn.ApplicationCall(
        app_id=self.upstream_pyth_app_id.value,
        app_args=(Bytes(b"get_price_unsafe"), feed_id),
        fee=0,
    ).submit()
    # PriceStruct ARC-4 encoding: price (8) | neg (1) | conf (8) | expo (8) | ...
    raw = result.last_log
    price = op.btoi(op.extract(raw, 0, 8))
    conf = op.btoi(op.extract(raw, 17, 8))
    expo = op.btoi(op.extract(raw, 25, 8))
    pubtime = op.btoi(op.extract(raw, 34, 8))
    return (price, expo, conf, pubtime)
```

```
@subroutine
```

```
def _read_folks(self, feed_id: Bytes) -> tuple[UInt64, bool]:
    if self.folks_oracle_app_id.value == 0:
        return (UInt64(0), False)
    result = itxn.ApplicationCall(
        app_id=self.folks_oracle_app_id.value,
        app_args=(Bytes(b"get_price"), feed_id),
        fee=0,
    ).submit()
    if result.num_logs == 0:
        return (UInt64(0), False)
    return (op.btoi(result.last_log), True)
```

```
@subroutine
```

```
def _read_dex_twap(self, feed_id: Bytes) -> tuple[UInt64, bool]:
    if self.bankon_dex_twap_app_id.value == 0:
        return (UInt64(0), False)
    result = itxn.ApplicationCall(
        app_id=self.bankon_dex_twap_app_id.value,
        app_args=(Bytes(b"get_twap_15m"), feed_id),
        fee=0,
    ).submit()
    if result.num_logs == 0:
        return (UInt64(0), False)
    return (op.btoi(result.last_log), True)
```

Client SDK

The SDK is a thin async Python wrapper covering reads, keeper loops, and subscription management. It uses `algokit-utils` v10 patterns and is compatible with the prior `PYTHAI.algo.pyth.oracle` SDK so a single application can consume both.

```
python
```

```

# sdk/pythai_bankon_oracle_sdk/reader.py
# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""Reader: spot, TWAP, median, SATPAY health."""
from __future__ import annotations
from dataclasses import dataclass
from algokit_utils import AlgorandClient

@dataclass(frozen=True)
class BankonPrice:
    spot: float
    median: float
    twap_1m: float
    twap_5m: float
    twap_15m: float
    twap_1h: float
    twap_24h: float
    publish_time: int
    num_sources: int

class BankonOracleReader:
    def __init__(self, algorand: AlgorandClient, app_id: int) -> None:
        self.algorand = algorand
        self.app_id = app_id

    async def price(self, feed_id: bytes, max_age: int = 60) -> BankonPrice:
        result = await self.algorand.app.call(
            app_id=self.app_id,
            method="get_price_no_older_than",
            args=[feed_id, max_age],
        )
        s = result.return_value
        expo = -s.expo
        scale = 10 ** expo
        return BankonPrice(
            spot=s.spot_price * scale,
            median=s.median_price * scale,
            twap_1m=s.twap_1m * scale,
            twap_5m=s.twap_5m * scale,
            twap_15m=s.twap_15m * scale,
            twap_1h=s.twap_1h * scale,
            twap_24h=s.twap_24h * scale,

```

```

        publish_time=s.publish_time,
        num_sources=s.num_sources,
    )

    async def satpay_health(self) -> dict:
        result = await self.algorand.app.call(
            app_id=self.app_id, method="get_satpay_health", args=[],
        )
        h = result.return_value
        return {
            "reserve_satoshis": h.reserve_satoshis,
            "circulating_bks": h.circulating_bks,
            "coverage_bps": h.coverage_bps,
            "coverage_pct": h.coverage_bps / 100.0,
            "healthy": h.healthy,
            "last_attestation": h.last_attestation,
        }

```

The keeper module runs an async loop that polls Hermes, submits PNAU updates to the upstream `PYTHAI.algo.pyth.oracle`, and then calls `BankonOracle.ingest()`. It is designed to run under systemd or as a Podman container with auto-restart, and its keeper key is held in a hardware HSM or an Algorand multisig with a dedicated keeper key.

FastAPI Surface

The API surface mirrors the upstream oracle but exposes the BANKON aggregations (median, TWAP, SATPAY health) and tier-gated access.

```
python
```

```

# api/mindx_bankon_api/app.py
# (c) 2026 BANKON — all rights reserved. Apache-2.0.
"""mindX BANKON Oracle API."""
from fastapi import FastAPI
from x402.http.middleware.fastapi import x402_payment_middleware
from x402.http.types import RouteConfig
from x402.http import PaymentOption
from x402.schemas import Network

from .routes_read import router as read_router
from .routes_subscribe import router as sub_router
from .routes_governance import router as gov_router

AVM_MAINNET: Network = "algorand:wGHE2Pwvdv7S12BL5Fa0P20EGYesN73ktiC1qzkkit8="
AERARIUM = "AERARIUM_TREASURY_ADDR_PLACEHOLDER"

app = FastAPI(title="mindX BANKON Oracle API", version="1.0.0")

metered_routes = {
    "GET /v1/bankon/price/{feed_id}": RouteConfig(
        accepts=[PaymentOption(
            scheme="exact", pay_to=AERARIUM, price="$0.01", network=AVM_MAINNET,
        )],
        description="Read a BANKON-aggregated price",
        mime_type="application/json",
    ),
    "GET /v1/bankon/twap/{feed_id}": RouteConfig(
        accepts=[PaymentOption(
            scheme="exact", pay_to=AERARIUM, price="$0.02", network=AVM_MAINNET,
        )],
        description="Read TWAP over a configurable window",
        mime_type="application/json",
    ),
}

app.add_middleware(x402_payment_middleware, routes=metered_routes)

app.include_router(read_router, prefix="/v1/bankon")
app.include_router(sub_router, prefix="/v1/bankon/subscribe")
app.include_router(gov_router, prefix="/v1/bankon/governance")

```

Tessera-tier subscribers receive a JWT signed by their Tessera key after a one-time subscription POST. Subsequent requests bearing that JWT bypass the x402 paywall — the JWT contains the subscription expiry and tier, signed offline from the AERARIUM-held verification key.

MCP Tools for Agentic Place

typescript

```
// agent/agentplace_tools.ts
// (c) 2026 BANKON — all rights reserved. Apache-2.0.
import { tool } from "ai";
import { z } from "zod";

const API = "https://mindx.pythai.net";

export const bankon_get_price = tool({
  description: "Get a BANKON-aggregated price (median across Pyth, Folks, DEX). Return price in USD.",
  parameters: z.object({
    symbol: z.string().describe("BTC/USD, ETH/USD, ALGO/USD, BKS/USD, BKPYP/USD, ..."),
    max_age_sec: z.number().default(60),
  }),
  execute: async ({ symbol, max_age_sec }) => {
    const r = await fetch(`${API}/v1/bankon/price/${encodeURIComponent(symbol)}?max_age_sec=${max_age_sec}`);
    return await r.json();
  },
});

export const bankon_get_twap = tool({
  description: "Get a time-weighted average price over a 60/300/900/3600/86400 second window.",
  parameters: z.object({
    symbol: z.string(),
    window_sec: z.number(),
  }),
  execute: async ({ symbol, window_sec }) => {
    const r = await fetch(`${API}/v1/bankon/twap/${encodeURIComponent(symbol)}?window_sec=${window_sec}`);
    return await r.json();
  },
});

export const bankon_satpay_health = tool({
  description: "Check the SATPAY reserve-versus-supply invariant. Returns coverage percentage.",
  parameters: z.object({}),
  execute: async () => {
    const r = await fetch(`${API}/v1/bankon/satpay/health`);
    return await r.json();
  },
});
```

Tests

python

```

# tests/test_synthetic_feeds.py
# (c) 2026 BANKON – all rights reserved. Apache-2.0.
import pytest
from algopy_testing import algopy_testing_context
from contracts.bankon_oracle import BankonOracle
from contracts.synthetic_feed_engine import (
    BKS_USD_BANKON, BTC_USD_PYTH, derive_bks_usd,
)

def test_bks_usd_derives_from_btc_with_8_decimal_shift():
    btc_price = 100_000 * 10**8    # 100,000 USD scaled by 10^8
    btc_expo_abs = 8
    bks_price, bks_expo = derive_bks_usd(btc_price, btc_expo_abs)
    assert bks_price == btc_price
    assert bks_expo == 16
    real_value = bks_price * 10**(-int(bks_expo))
    assert abs(real_value - 0.001) < 1e-9    # 100,000 / 10^8

def test_bks_supply_invariant():
    from contracts.synthetic_feed_engine import BKS_SUPPLY, SATS_PER_BTC
    assert BKS_SUPPLY.value == 21_000_000 * SATS_PER_BTC.value

def test_ingest_bks_inherits_btc_confidence():
    with algopy_testing_context() as ctx:
        oracle = BankonOracle()
        # Seed BTC price.
        from contracts.bankon_price_struct import BankonPriceStruct
        from algopy import arc4
        oracle.prices[BTC_USD_PYTH] = BankonPriceStruct(
            spot_price=arc4.UInt64(100_000 * 10**8),
            spot_negative=arc4.Bool(False),
            median_price=arc4.UInt64(100_000 * 10**8),
            median_negative=arc4.Bool(False),
            twap_1m=arc4.UInt64(100_000 * 10**8),
            twap_5m=arc4.UInt64(100_000 * 10**8),
            twap_15m=arc4.UInt64(100_000 * 10**8),
            twap_1h=arc4.UInt64(100_000 * 10**8),
            twap_24h=arc4.UInt64(100_000 * 10**8),
            expo=arc4.UInt64(8),
            expo_negative=arc4.Bool(True),

```

```

        publish_time=arc4.UInt64(1_700_000_000),
        conf=arc4.UInt64(50 * 10**8), # $50 confidence interval
        source_bitmap=arc4.UInt64(1),
        num_sources=arc4.UInt64(1),
    )
    oracle._ingest_bks()
    bks = oracle.prices[BKS_USD_BANKON]
    assert bks.conf.native == 50 * 10**8

```

The full test matrix covers:

- `test_synthetic_feeds.py`: BKS derivation correctness, BKPYP composition with USDC peg drift, BKPYP DEX TWAP absence handling, BKS supply invariant.
- `test_twap.py`: ring-buffer overflow, eviction of oldest entries, TWAP computation across discontinuous timestamps, empty buffer return value, single-sample TWAP equals spot.
- `test_median.py`: median-of-three correctness on all 6 permutations, two-source mean fallback, single-source pass-through, zero-source assertion failure.
- `test_satpay_health.py`: full reserve coverage returns 10,000 bps and healthy=True; 50% coverage returns 5,000 bps and unhealthy; zero circulating supply returns 10,000 bps and healthy; coverage just below threshold triggers unhealthy.
- `test_governance.py`: only Senatus can pause; rotation correctness; non-Senatus call raises SENATUS_ONLY; rotation atomic with multisig.
- `test_end_to_end.py`: a keeper submits a Pyth update, calls `ingest()`, the spot/median/TWAP fields populate, the BKS synthetic price is computed, the SATPAY health refreshes, the deviation alert fires when injected, the registered downstream consumer's `on_deviation_alert` receives the inner-app-call.

Deployment

python

```

# deploy/deploy_mainnet.py
# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""Mainnet deployment, invoked by the Senatus 5-of-7 multisig."""
import os
from algokit_utils import AlgorandClient, get_account
from algopy import arc4

from contracts.bankon_oracle import BankonOracle

UPSTREAM_PYTH_APP_ID = int(os.environ["UPSTREAM_PYTH_APP_ID"])
FOLKS_ORACLE_APP_ID = int(os.environ.get("FOLKS_ORACLE_APP_ID", "0"))
BANKON_DEX_TWAP_APP_ID = int(os.environ["BANKON_DEX_TWAP_APP_ID"])
BKS_ASA_ID = int(os.environ["BKS_ASA_ID"])
TESSERA_ASA_ID = int(os.environ["TESSERA_ASA_ID"])
AERARIUM_ADDR = os.environ["AERARIUM_ADDR"]
SENATUS_ADDR = os.environ["SENATUS_ADDR"]

def main() -> None:
    algorand = AlgorandClient.from_environment()
    deployer = get_account("DEPLOYER", algorand)
    client = algorand.client.from_algorand_python(BankonOracle, deployer)
    app = client.create()
    client.send.set_upstream_pyth(arc4.UInt64(UPSTREAM_PYTH_APP_ID))
    if FOLKS_ORACLE_APP_ID:
        client.send.set_folks_oracle(arc4.UInt64(FOLKS_ORACLE_APP_ID))
    client.send.set_dex_twap(arc4.UInt64(BANKON_DEX_TWAP_APP_ID))
    client.send.set_aerarium(arc4.Address(AERARIUM_ADDR))
    client.send.rotate_senatus(arc4.Address(SENATUS_ADDR))
    print(f"PYTHAI.bankon.oracle deployed: app_id={app.app_id}")
    print(f"  upstream pyth: {UPSTREAM_PYTH_APP_ID}")
    print(f"  dex twap:      {BANKON_DEX_TWAP_APP_ID}")
    print(f"  bks asa:       {BKS_ASA_ID}")
    print(f"  tessera asa:   {TESSERA_ASA_ID}")
    print(f"  aerarium:      {AERARIUM_ADDR}")
    print(f"  senatus:       {SENATUS_ADDR}")

if __name__ == "__main__":
    main()

```

The deployment sequence is: deploy on TestNet against the TestNet upstream

`PYTHAI.algo.pyth.oracle`; run a 14-day beta with synthetic Hermes captures; commission an

Adevar audit (the same firm that audited Folks Finance's Algorand Wormhole NTT in October 2025, which is the closest body of relevant prior art); deploy on MainNet via Senatus 5-of-7 with the audited bytecode hash pinned in the NFD record; register the app identifier in `agenticplace.pythai.net/allchain.html` and the NFD `bankon.oracle` under the `PYTHAI` parent.

Operational Runbook

The keeper runs continuously and submits the major-feed batch (BTC, ETH, ALGO, USDC, USDT, SOL, AVAX, MATIC) every 5 seconds. The synthetic feeds BKS and BKPYP are computed inside the same `ingest()` call so no separate keeper is required for them. The SATPAY reserve attester runs every 15 minutes on a separate cron, calling `set_satpay_reserves` with the latest off-chain attested Bitcoin reserve count. The deviation-alert dispatcher is purely on-chain and fires inside `ingest()` whenever spot deviates from 5-minute TWAP by more than `alert_threshold_bps`. The Senatus 5-of-7 holds the only authority to pause, change upstream pointers, modify thresholds, or rotate itself.

In the event of an upstream oracle compromise or Pyth Network outage, the contract continues to serve the last-known good price for `default_max_age` seconds, after which `get_price_no_older_than` reverts and downstream consumers fail safely. The Senatus may activate `set_pause(true)` to halt all reads and ingests during incident response.

Caveats

The Pyth Entropy contract has not shipped on Algorand as of May 2026; until it does, the keeper selection mechanism falls back to a Senatus-rotated seed combined with `Global.latest_timestamp`. When Pyth Entropy ships, replace the seed source via Senatus governance; no contract code change is required.

The Folks Finance oracle integration assumes a specific ABI shape (`get_price(feed_id) → log`) that has not been independently verified against the production Folks Finance oracle application. Before mainnet, confirm the ABI from the audited Folks contracts and adjust `_read_folks` to match. If Folks ships a v2 oracle with a different signature, the function is the only place that needs updating.

BKPYP/USD relies on a BANKON-controlled DEX TWAP. The current BANKON architecture uses SpinTrade as the canonical DEX; the DEX TWAP application identifier must be set via `set_dex_twap` before BKPYP synthesis returns meaningful values. Until BKPYP trades on a sufficiently liquid pair with continuous price discovery, the BKPYP/USD feed should be marked `source_bitmap` bit 3 (synthetic) and consumed with appropriate skepticism by downstream systems.

The ARC-4 encoding of `BankonPriceStruct` is sensitive to field ordering and Algopy version. Pin `algorand-python >= 3.5.0, < 4.0.0` in `pyproject.toml`. Any minor-version bump in Algopy may shift the ABI layout and require redeployment with a migration that re-reads the previous boxes and writes them under the new encoding.

The `_read_upstream_pyth` helper assumes the upstream `PYTHAI.algo.pyth.oracle` exposes `get_price_unsafe(feed_id) → log` returning ARC-4 encoded `PriceStruct`. The offsets used in the byte-extraction (0, 17, 25, 34) presume the field ordering documented in the upstream `contracts/price_feed_storage.py`. If the upstream encoding changes, this helper must be updated in lockstep. The safer long-term pattern is to encode the upstream return via a typed ABI client generated by `algokit generate client`; this is recommended for production.

Keeper bounty payments are funded from the AERARIUM treasury via inner-payment; the deployment script must explicitly opt the application account into receiving Algos and fund it with enough microAlgos to cover at least 30 days of expected ingestion calls at the configured bounty rate. The Senatus should monitor this balance and top up quarterly.

The `set_satpay_reserves` method is the single most security-critical entry point because it determines whether the SATPAY peg appears healthy. The Senatus 5-of-7 gate is mandatory; do not relax this requirement under any circumstances. Consider additionally encoding a maximum allowed delta per call (e.g. reserve changes greater than 10% in a single call require a second confirmation transaction from a different Senatus subset within a 1-hour window).